



**Universidad
Nacional de
General
Sarmiento**

Introducción a la Programación Pokedex

Com. 02 - 2025

Alumnos: Almanza, Rodrigo
Bulgach, Priscila

Profesores: Bottino, Flavia
Bidart Gauna, Lucas

Introducción

Este trabajo práctico tiene como objetivo el desarrollo de una aplicación web interactiva que permite a los usuarios buscar, visualizar y explorar información sobre Pokémon. El proyecto combina diseño, funcionalidad y acceso a datos web, ofreciendo una experiencia visual atractiva y dinámica.

La aplicación está construida utilizando Django, un framework de desarrollo web que permite estructurar el proyecto de manera ordenada utilizando una arquitectura por capas. Los datos son recuperados de una API externa, que luego se presentan al usuario en forma de tarjetas que muestran la información de los Pokémon de manera simple y atractiva.

Para llevar a cabo el desarrollo, se utilizaron herramientas como Visual Studio Code, GitHub para la gestión del código en equipo, y Python como lenguaje de programación de la lógica de negocio. Además, se trabajó con una base de datos local en formato SQLite, que permite guardar información.

Capa de Presentación

Como el trabajo está organizado en capas, nos pareció pertinente presentar cada capa y mencionar las modificaciones realizadas en cada una de aquellas. La capa de presentación es la encargada de interactuar con el usuario final. Muestra datos y recoge las acciones del usuario (clics, formularios, etc.). En la aplicación de Pokedex, esta capa está desarrollada en HTML. Y utilizando el Framework Django se conecta con la lógica de negocios y las capas de persistencia y base de datos.

home.html

```
{% for img in images %}
    <div class="col">
        <!-- evaluar si la imagen pertenece al tipo fuego, agua o
planta -->
        {% if "fire" in img.types %}
            <div class="card border-danger mb-3 ms-5"
style="max-width: 540px; ">
                {% elif "water" in img.types %}
                    <div class="card border-primary mb-3 ms-5"
style="max-width: 540px; ">
                        {% elif "grass" in img.types %}
                            <div class="card border-success mb-3 ms-5"
style="max-width: 540px; ">
                                {% else %}
                                    <div class="card border-warning mb-3 ms-5"
```

```
style="max-width: 540px; ">
    {% endif %}
    ...
<\div>
{% endfor %}
```

A la hora de modificar la plantilla que se utiliza al renderizar la galería donde se muestran las imágenes de los pokémon, encontramos dificultades para evitar la repetición de código. Aunque en el grupo surgieron algunas ideas para solucionarlo, ninguna se ajustaba a los parámetros de tiempo o simplicidad que requería el trabajo. Este problema de repetición se volvió realmente evidente al agregar una gran variedad de tipos, para filtrar las tarjetas visibles en pantalla. Dejando de lado ese problema logramos implementar la lógica que permite filtrar las tarjetas de los Pokemon por tipo. Esto se logró gracias a la utilización de la estructura `if`, que nos permitió filtrar aquellas tarjetas cuyo tipo eran de un tipo específico.

Capa de Negocio

La capa de negocio contiene la lógica que da sentido a la aplicación: reglas, validaciones, decisiones, transformaciones de datos, etc. Define cómo debe funcionar el sistema según los requerimientos del dominio. En la aplicación de la Pokedex esta capa se ve representada por ejemplo en `services.py`, `translator.py`, `card.py`.

```
getAllImages() :

    list_of_cards = []
    for raw_image in transport.getAllImages():
        card = translator.fromRequestIntoCard(raw_image)
        list_of_cards.append(card)

    return list_of_cards
```

La modificación de la función `getAllImages`, fue nuestro primer desafío. Si bien rápidamente logramos recuperar la información de los pokémon utilizando una función del módulo `transport`. Al renderizar la página web, en las tarjetas se mostraba información errónea. Esto sucedió porque enviamos a renderizar datos crudos sin una previa transformación a un tipo de dato adecuado. Pero aquello que al comienzo se presentó como una traba, terminó siendo beneficioso, ya que nos permitió ver cómo estaban estructurados los datos sobre los pokemon. Lo que resultó muy útil en la resolución de los sucesivos problemas. Finalmente logramos convertir los datos provenientes de la API utilizando la función

`fromRequestIntoCard()` del modulo `translator`. Al realizar estas modificaciones la función `getAllImages()` quedó funcional.

Filtros

Al identificar lo trabajoso que era, en esta etapa del desarrollo, abordar el problema de los favoritos. Nos enfocamos en la implementación de los filtros. Esta etapa del trabajo no presentó mayores problemas, gracias al entendimiento de los atributos de las `Card`, logrado en etapas previas del desarrollo. Lo único necesario para desarrollar estas funciones, es obtener las imágenes de los Pokémon utilizando la función `getAllImages()`. Luego, utilizando la estructura de bucle se recorre cada una de las tarjetas almacenando solo aquellas que tienen el nombre o el tipo de Pokémon que se pase como argumento.

Para el filtro por tipo, decidimos añadir un nuevo atributo a las tarjetas de cada pokemon. Este nuevo atributo contiene una lista con las `urls` que apuntan a las imágenes representativas del tipo de Pokémon. Al implementar este nuevo atributo, fuimos capaces de renderizar los tipos de los Pokémon con un estilo mucho más bonito.

Favoritos

En esta etapa del trabajo, fue necesario familiarizarnos con los objetos `HttpRequest` y `HttpResponse`. Si bien como grupo logramos comprender sus aspectos fundamentales, incluso después de finalizar el proyecto aún tenemos algunas dudas sobre su funcionamiento. Esta fase fue la más demandante en cuanto a tiempo, ya que requirió una extensa investigación por parte del equipo sobre el funcionamiento de Django.

Capa de Aplicación

La capa de aplicación controla el flujo de la aplicación. Recibe las peticiones del usuario desde la presentación y coordina las llamadas a las capas inferiores (negocio, datos). En nuestro caso particular corresponde al archivo `views.py`.

```
home(request):  
  
    images = services.getAllImages()  
    favourite_list = []  
  
    for img_card in services.getAllFavourites(request):  
        favourite_list.append(img_card.name)
```

```
return render(request, 'home.html', { 'images': images,
    'favourite_list': favourite_list })
```

Para implementar esta función, fue necesario reutilizar funciones desarrolladas previamente. Utilizando la notación de punto, accedimos a una lista de tarjetas con la información de los Pokémon. Posteriormente, solo fue necesario un bucle `for` para completar la lista de favoritos con los nombres de los Pokémon que ya estaban marcados como favoritos.

Capa de Persistencia Y Base de Datos

Se encarga de leer y escribir datos en la base de datos de forma desacoplada del resto del sistema. Traduce objetos del programa a estructuras almacenables (y viceversa). Ambas capas estaban enteramente desarrolladas y no requirió ninguna modificación.

Conclusiones

Por el lado positivo, creemos que a lo largo del proyecto, se promueve la colaboración entre integrantes del equipo, el uso responsable de herramientas de control de versiones, y la implementación de buenas prácticas de programación que permiten construir un sistema claro, funcional y mantenible. Aunque es importante señalar que por otro lado, durante el desarrollo del trabajo. También nos enfrentamos a ciertas dificultades, como la escasa documentación disponible en español. Además de la ausencia de una fuente de información unificada para resolver dudas relacionadas con GitHub, Bootstrap, HTML, JSON y Django que complicó innecesariamente el progreso del proyecto.